

```

1 terraform {
2     required_providers {
3         docker = {
4             source = "kreuzwerker/docker"
5             version = "~> 3.0.2"
6         }
7     }
8 }
9
10 provider "docker" {}
11 resource "docker_image" "todo-service" {
12     name="phoeng/todo:3.0"
13     keep_locally=false
14 }
15
16 resource "docker_image" "redis" {
17     name="redis"
18     keep_locally=false
19 }
20
21 resource "docker_image" "todo-noti" {
22     name="phoeng/todo-notification:1.1"
23     keep_locally = false
24 }
25
26 resource "docker_network" "todo_net" {
27     name = "todo_network"
28 }
29
30 resource "docker_container" "todo-service" {
31     image=docker_image.todo-service.image_id
32     name="todo-service"
33     ports{
34         internal=8000
35         external=8000
36     }
37     env = [
38         "REDIS_HOST=redis",
39         "REDIS_PORT=6379",
40         "NOTIFICATION_HOST=notification"
41     ]
42     networks_advanced {
43         name = docker_network.todo_net.name
44     }
45 }
46
47 resource "docker_container" "redis" {
48     image=docker_image.redis.image_id
49     name="redis"
50     ports{
51         internal=6379
52         external=6379
53     }
54     networks_advanced {
55         name = docker_network.todo_net.name
56     }
57 }
58
59 resource "docker_container" "notification" {
60     image = docker_image.todo-noti.image_id
61     name = "notification"
62     ports {
63         internal = 9000
64         external = 9000
65     }
66     networks_advanced {
67         name = docker_network.todo_net.name
68     }
69 }
70
71 output "todo_id" {
72     value = docker_container.todo-service.id
73 }
74
75 output "todo_addr" {
76     value = docker_container.todo-service.network_data[0].ip_address
77 }
78
79 output "redis_id" {
80     value = docker_container.redis.id
81 }
82
83 output "redis_addr" {
84     value = docker_container.redis.network_data[0].ip_address
85 }
86
87 output "noti_id" {
88     value = docker_container.notification.id
89 }
90
91 output "noti_addr" {
92     value = docker_container.notification.network_data[0].ip_address
93 }
94

```

Step 1: Terraform init

```
(base) PS C:\Users\USER\Desktop\Software_def\act5> terraform init -upgrade
Initializing the backend...
Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "~> 3.0.2"...
- Installing kreuzwerker/docker v3.0.2...
- Installed kreuzwerker/docker v3.0.2 (self-signed, key ID BD080C4571C6104C)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
(base) PS C:\Users\USER\Desktop\Software_def\act5>
```

Step 2: Terraform plan

```
(base) PS C:\Users\USER\Desktop\Software_def\act5> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# docker_container.redis will be created
+ resource "docker_container" "redis" {
  + attach                = false
  + bridge                = (known after apply)
  + command               = (known after apply)
  + container_logs        = (known after apply)
  + container_read_refresh_timeout_milliseconds = 15000
  + endpoint              = (known after apply)
  + env                  = (known after apply)
  + exit_code             = (known after apply)
  + hostname              = (known after apply)
  + id                    = (known after apply)
  + image                 = (known after apply)
  + init                  = (known after apply)
  + ipc_mode              = (known after apply)
  + log_driver            = (known after apply)
  + logs                  = false
  + must_run              = true
  + name                  = "redis-todo"
  + network_data          = (known after apply)
  + read_only             = false
  + remove_volumes       = true
  + restart               = "no"
  + rm                    = false
  + runtime               = (known after apply)
}
```

Step 3: terraform apply

```
~ redis_addr = "172.17.0.3" -> (known after apply)
~ redis_id   = "05719cec5a2fea27163b78eceb6e4c1331df51ff86600f349778dd295554a5a1" -> (known after apply)
~ todo_addr  = "172.17.0.4" -> (known after apply)
~ todo_id    = "790e770d958868e121ec6c6c43e607ab008a9da02d285bee6d96d41331e7cad5" -> (known after apply)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

docker_container.todo-service: Destroying... [id=790e770d958868e121ec6c6c43e607ab008a9da02d285bee6d96d41331e7cad5]
docker_container.redis: Destroying... [id=05719cec5a2fea27163b78eceb6e4c1331df51ff86600f349778dd295554a5a1]
docker_container.notification: Destroying... [id=98bf31bf6a49b3d0ca8ca90491d1934c72b40e5d22d90523a38fd644ed54a06d]
docker_container.redis: Destruction complete after 1s
docker_container.redis: Creating...
docker_container.todo-service: Destruction complete after 1s
docker_container.todo-service: Creating...
docker_container.notification: Destruction complete after 2s
docker_container.notification: Creating...
docker_container.redis: Creation complete after 1s [id=ae24ff4613538992a29575c4f5bd0a5a66f5abc774bd45e745a7a3280aec43c1]
docker_container.todo-service: Creation complete after 2s [id=be1046b3284faa3518fddfc231d1871134307d4d4488dd28c753631f5effdad9]
docker_container.notification: Creation complete after 1s [id=1f0866d6e120d394c2a285eb110b9c213a07457e43ed7d896a0c89832d8ba286]

Apply complete! Resources: 3 added, 0 changed, 3 destroyed.

Outputs:

noti_addr = "172.17.0.4"
noti_id   = "1f0866d6e120d394c2a285eb110b9c213a07457e43ed7d896a0c89832d8ba286"
redis_addr = "172.17.0.2"
redis_id   = "ae24ff4613538992a29575c4f5bd0a5a66f5abc774bd45e745a7a3280aec43c1"
todo_addr  = "172.17.0.3"
todo_id    = "be1046b3284faa3518fddfc231d1871134307d4d4488dd28c753631f5effdad9"
```

```

• (base) PS C:\Users\USER\Desktop\Software_def\act5> .\activity5-CP_windows_amd64.exe
2026/09/08 17:15:16 Current Timestamp: 2026-09-08T17:15:16+07:00
👉 Enter [REQUIRED] Terraform file path (.tf) (leave blank for 'main.tf'):
2026/09/08 17:15:17 ✅ / Terraform file path is exist.
2026/09/08 17:15:17 ✅ / Terraform is installed.
2026/09/08 17:15:17 ✅ / Terraform is initialized.
2026/09/08 17:15:20 ✅ / Terraform plan is generated.
2026/09/08 17:15:20 ✅ / Container todo-service exists.
2026/09/08 17:15:20 ✅ / Container redis exists.
2026/09/08 17:15:20 ✅ / All specified containers are running.
2026/09/08 17:15:20 ✅ / URL http://localhost:8000 returns 200.
2026/09/08 17:15:20 ✅ / Todo is up and running at http://localhost:8000
2026/09/08 17:15:20 ✅ / POST request successful
2026/09/08 17:15:20 ✅ / POST request to http://localhost:8000 was successful.
2026/09/08 17:15:20 ✅ / GET request shows result from previous POST request to http://localhost:8000.
2026/09/08 17:15:20 Result: true
2026/09/08 17:15:20 🎉 Looks good! Please enter your StudentID and Full name below
👉 StudentID: 6630199021
👉 Full Name (TH): เพลลาภ กสิณะ
2026/09/08 17:15:28 🎉 Successfully submitted. Your lucky number is 20277615304696972
2026/09/08 17:15:28 🎉🎉🎉 Congratulations! You have completed the activity 🎉🎉🎉
2026/09/08 17:15:28 ⚠️ Don't forget! you still need to submit your assignment via MyCourseVille ⚠️
❖ (base) PS C:\Users\USER\Desktop\Software_def\act5>

```

Step 4: terraform destroy

```

(base) PS C:\Users\USER\Desktop\Software_def\act5> terraform destroy
}
}

Plan: 0 to add, 0 to change, 7 to destroy.

Changes to Outputs:
- noti_addr = "172.17.0.4" -> null
- noti_id = "1f0866d6e120d394c2a285eb110b9c213a07457e43ed7d896a0c89832d8ba286" -> null
- redis_addr = "172.17.0.2" -> null
- redis_id = "ae24ff4613538992a29575c4f5bd0a5a66f5abc774bd45e745a7a3280aec43c1" -> null
- todo_addr = "172.17.0.3" -> null
- todo_id = "be1046b3284faa3518fddfc231d1871134307d4d4488dd28c753631f5effdad9" -> null

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

docker_container.notification: Destroying... [id=1f0866d6e120d394c2a285eb110b9c213a07457e43ed7d896a0c89832d8ba286]
docker_container.redis: Destroying... [id=ae24ff4613538992a29575c4f5bd0a5a66f5abc774bd45e745a7a3280aec43c1]
docker_container.todo-service: Destroying... [id=be1046b3284faa3518fddfc231d1871134307d4d4488dd28c753631f5effdad9]
docker_container.redis: Destruction complete after 1s
docker_image.redis: Destroying... [id=sha256:298e5b3bc566bade82f46ad5511777a4a07a294097ce16ada2f6a42be5239df5redis]
docker_container.notification: Destruction complete after 1s
docker_image.todo-noti: Destroying... [id=sha256:b4dfc5709b001be6e5b8aa787acd273185e0adf990f9061e39a821638fe7b7c1phoeng/todo-notific
docker_image.redis: Destruction complete after 1s
docker_image.todo-noti: Destruction complete after 1s
docker_container.todo-service: Destruction complete after 2s
docker_image.todo-service: Destroying... [id=sha256:700cfd1376850be5e3224e0766d371b74b9ac8e15adc290a309cb2b9e8e3dfe2phoeng/todo:3.0]
docker_network.todo_net: Destroying... [id=f71eb9b35b097cf01f98d6057a068e5562cfffed2159fad50801bcc2f64e4008]
docker_image.todo-service: Destruction complete after 1s
docker_network.todo_net: Destruction complete after 3s

Destroy complete! Resources: 7 destroyed.
(base) PS C:\Users\USER\Desktop\Software_def\act5>

```